

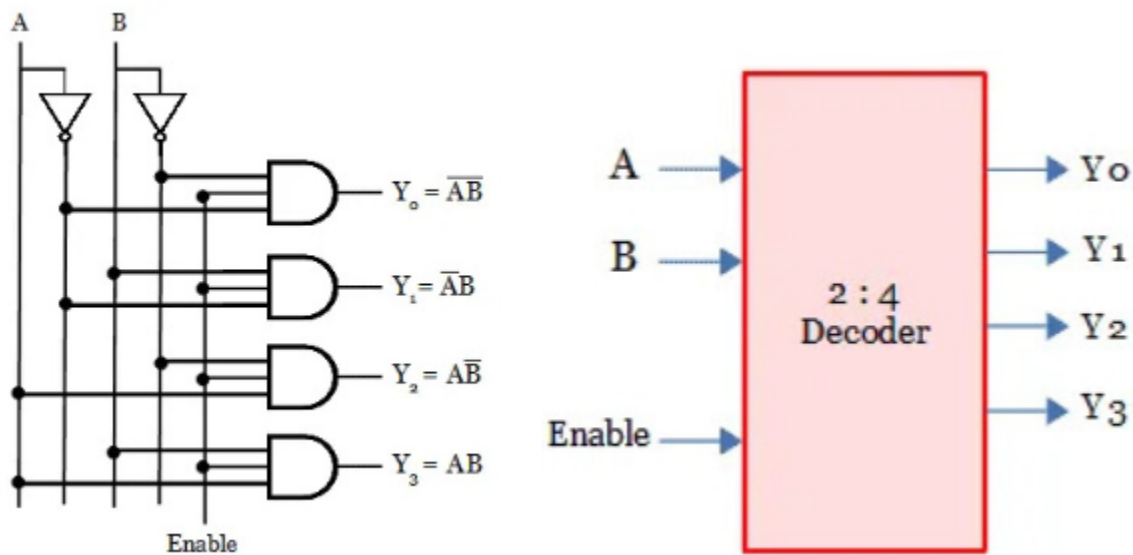
Circuit Simulation Lab:6

Decoders

The purpose of this experiment is to introduce the design of simple combinational circuits, in this case Decoders. A decoder is a multiple-input, multiple-output combinational logic circuit. It converts the n bit data inputs into the coded 2^n outputs. It decodes the information hidden by the encoder.

Software tools Requirement

Icarus & gtkwave



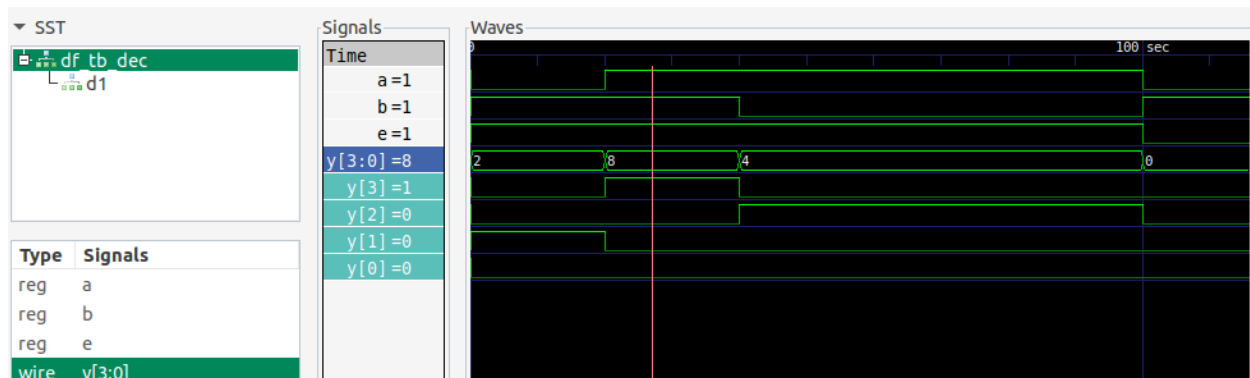
EN	A	B	Y_3	Y_2	Y_1	Y_0
0	x	x	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

DATAFLOW :

```
module df_dec(  
input a , b ,e,  
output [3:0] y );
```

```
assign y[3] = e & (a & b);  
    assign y[2] = e & (a & ~b);  
    assign y[1] = e & (~a & b);  
    assign y[0] = e & (~a & ~b);
```

```
endmodule
```



```
vlsi_dev_box@vlsi-box:~/Documents/NIELIT_ASSIGNMENTS/EXP_NO_6$ vvp out  
VCD info: dumpfile df_dec.vcd opened for output.  
a = 0 , b = 1 , e = 1 , y = 0010  
a = 1 , b = 1 , e = 1 , y = 1000  
a = 1 , b = 0 , e = 1 , y = 0100  
a = 1 , b = 0 , e = 1 , y = 0100  
a = 1 , b = 0 , e = 1 , y = 0100  
a = 0 , b = 1 , e = 0 , y = 0000
```

BEHAVIOURAL:

```
module bm_dec(  
    input wire a,  
    input wire b,  
    input wire e,  
    output reg [3:0] y  
);
```

```
always @(*) begin  
    if (!e) begin  
        y = 4'b0000; // Output is disabled  
    end else begin  
        case ({a, b})
```

```

        2'b00: y = 4'b0001;
        2'b01: y = 4'b0010;
        2'b10: y = 4'b0100;
        2'b11: y = 4'b1000;
        default: y = 4'b0000;
    endcase
end
end

```

```
Endmodule
```

```
`timescale 1ns / 1ps
```

```
module tb_bm_dec;
```

```

    reg a;
    reg b;
    reg e;
    wire [3:0] y;

```

```

bm_dec uut (
    .a(a),
    .b(b),
    .e(e),
    .y(y)
);

```

```
initial begin
```

```

    $dumpfile("bm_dec.vcd");
    $dumpvars(0, tb_bm_dec);

```

```
$monitor("Time=%0t | e=%b a=%b b=%b | y=%b", $time, e, a, b, y);
```

```
// Case 1: Enable is LOW (Outputs must be 0000)
```

```

e = 0; a = 0; b = 0; #20;
e = 0; a = 1; b = 1; #20;

```

```
// Case 2: Enable is HIGH (Normal Decoder Operation)
```

```

e = 1; a = 0; b = 0; #20;
e = 1; a = 0; b = 1; #20;
e = 1; a = 1; b = 0; #20;
e = 1; a = 1; b = 1; #20;

```

```

    $finish;
end

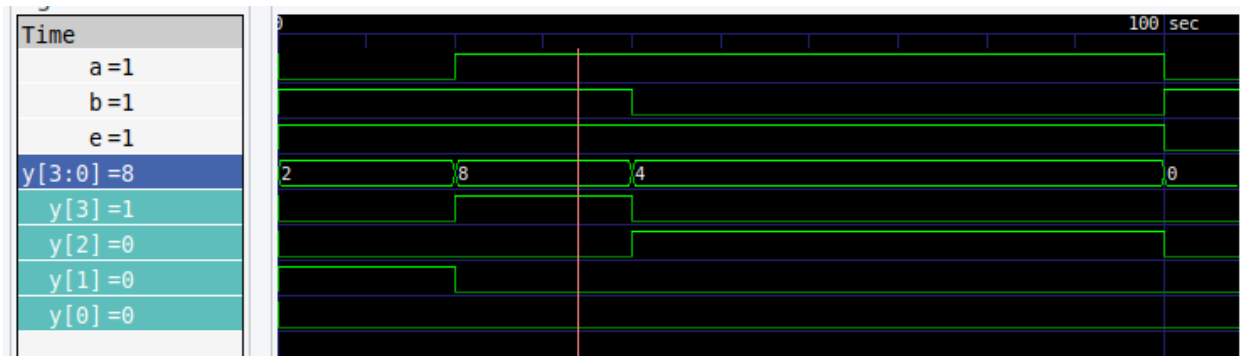
```

Endmodule

```

vlsi_dev_box@vlsi-box:~/Documents/NIELIT_ASSIGNMENTS/EXP_NO_6$ gedit tb_b
vlsi_dev_box@vlsi-box:~/Documents/NIELIT_ASSIGNMENTS/EXP_NO_6$ iverilog b
tb_bm_dec.v
vlsi_dev_box@vlsi-box:~/Documents/NIELIT_ASSIGNMENTS/EXP_NO_6$ vvp out
VCD info: dumpfile df_dec.vcd opened for output.
a = 0 , b = 1 , e = 1 , y = 0010
a = 1 , b = 1 , e = 1 , y = 1000
a = 1 , b = 0 , e = 1 , y = 0100
a = 1 , b = 0 , e = 1 , y = 0100
a = 1 , b = 0 , e = 1 , y = 0100
a = 0 , b = 1 , e = 0 , y = 0000

```



STRUCTURAL

```

module sm_dec(
    input wire a,
    input wire b,
    input wire e,
    output wire [3:0] y
);

```

```

    wire not_a;
    wire not_b;

```

```

    wire term0, term1, term2, term3;

```

```

    not gate_na (not_a, a);
    not gate_nb (not_b, b);

```

```
and gate_t0 (term0, not_a, not_b); // a=0, b=0
and gate_t1 (term1, not_a, b);    // a=0, b=1
and gate_t2 (term2, a, not_b);    // a=1, b=0
and gate_t3 (term3, a, b);        // a=1, b=1
```

```
and gate_y0 (y[0], term0, e);
and gate_y1 (y[1], term1, e);
and gate_y2 (y[2], term2, e);
and gate_y3 (y[3], term3, e);
```

```
Endmodule
```

```
`timescale 1ns / 1ps
```

```
module tb_sm_dec;
```

```
    reg a;
    reg b;
    reg e;
    wire [3:0] y;
```

```
    sm_dec uut (
        .a(a),
        .b(b),
        .e(e),
        .y(y)
    );
```

```
    initial begin
```

```
        $dumpfile("sm_dec.vcd");
        $dumpvars(0, tb_sm_dec);
```

```
        $monitor("Time=%0t | e=%b a=%b b=%b | y=%b", $time, e, a, b, y);
```

```
        e = 0; a = 0; b = 0; #20;
        e = 0; a = 1; b = 1; #20;
```

```

e = 1; a = 0; b = 0; #20;
e = 1; a = 0; b = 1; #20;
e = 1; a = 1; b = 0; #20;
e = 1; a = 1; b = 1; #20;

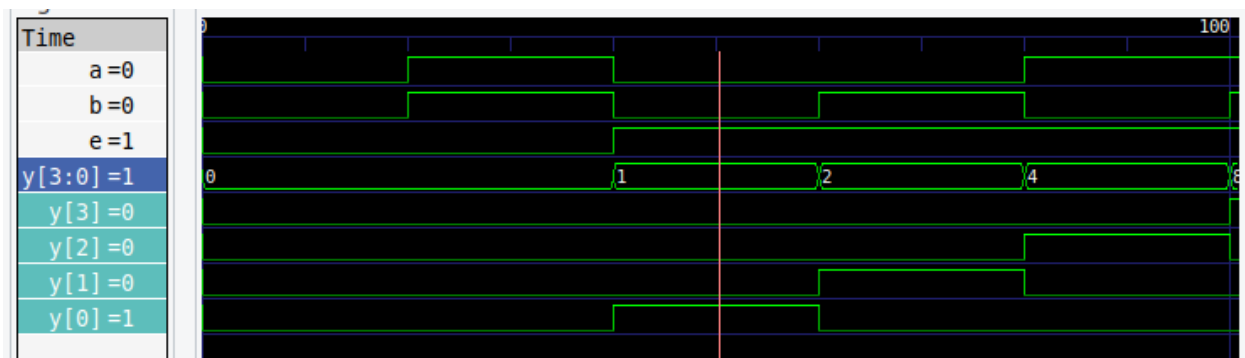
```

```

$finish;
end

```

```
Endmodule
```



Design a 4 to 16 decoder using case statements in Verilog

```

module dec_4x16(
    input wire [3:0] a,
    output reg [15:0] b
);

```

```

always @(*) begin
    case (a)
        4'd0 : b = 16'h0001;
        4'd1 : b = 16'h0002;
        4'd2 : b = 16'h0004;
        4'd3 : b = 16'h0008;
        4'd4 : b = 16'h0010;
        4'd5 : b = 16'h0020;
        4'd6 : b = 16'h0040;
        4'd7 : b = 16'h0080;
        4'd8 : b = 16'h0100;

```

```

4'd9 : b = 16'h0200;
4'd10 : b = 16'h0400;
4'd11 : b = 16'h0800;
4'd12 : b = 16'h1000;
4'd13 : b = 16'h2000;
4'd14 : b = 16'h4000;
4'd15 : b = 16'h8000;
default: b = 16'h0000;
endcase
end

```

Endmodule

